

The University of Faisalabad

Department of Computer Science



Name: Humna Imran

Registration Number: 2023-BS-AI-017

Instructor: Samraiz Zahid

Programming for AI

CS-302 Lab

BS Artificial Intelligence

Table of Content

LAB # 1	3
LAB # 2.....	10
LAB # 3.....	17
LAB # 4	23
LAB # 5.....	28
LAB # 8.....	34
LAB # 9.....	40
LAB # 10.....	46
LAB # 11.....	52

LAB # I

Python Fundamentals - Control Flow & Data Structures

Question # I:

Simulate a bank ATM. The user enters a 4-digit PIN, and the system checks if it is valid. It displays the account balance and asks for a withdrawal amount. If the PIN is valid, the system deducts the amount and updates the balance. If there are 3 or more invalid attempts, the card is blocked. The system also computes service charges (2% per withdrawal).

Code:

```
print("---- Welcome to Humna Imran's Banking System ----")

# Setup initial account details
correct_pin = 1234
current_balance_humna = 50000 # Initial balance
attempts = 0
max_attempts = 3
card_blocked = False

# Loop to check PIN attempts
while attempts < max_attempts:
    entered_pin = int(input("Please enter your 4-digit PIN: "))

    if entered_pin == correct_pin:
        print(f"Login Successful! Current Balance: Rs.
{current_balance_humna}")

        # Ask for withdrawal
        withdraw_amount = float(input("Enter withdrawal amount: "))

        # Calculate Service Charge (2%)
        service_charge = withdraw_amount * 0.02
        total_deduction = withdraw_amount + service_charge

        # Check if balance is sufficient
        if total_deduction <= current_balance_humna:
            current_balance_humna = current_balance_humna -
total_deduction
            print(f"Withdrawal: Rs. {withdraw_amount}")
```

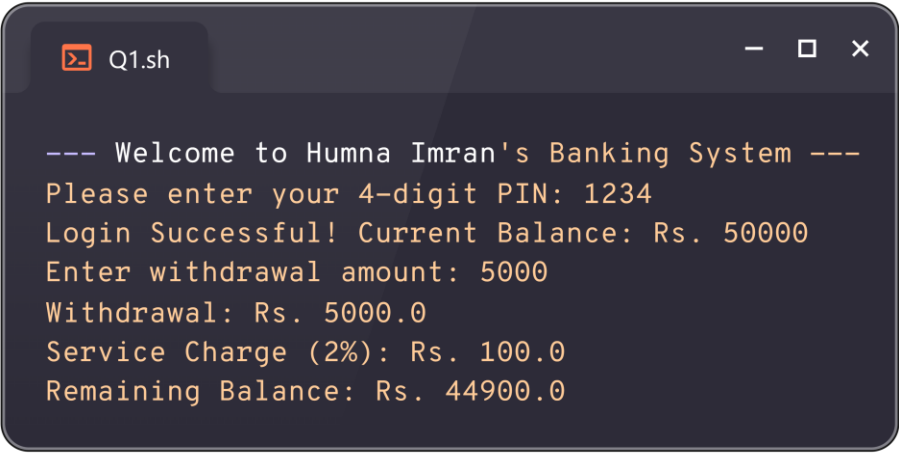
```

        print(f"Service Charge (2%): Rs. {service_charge}")
        print(f"Remaining Balance: Rs. {current_balance_humna}")
    else:
        print("Insufficient Funds!")
        break
    else:
        attempts += 1
        print(f"Incorrect PIN. Attempts remaining: {max_attempts - attempts}")

if attempts == max_attempts:
    print("Card Blocked! Too many invalid attempts.")

```

output



```

Q1.sh

--- Welcome to Humna Imran's Banking System ---
Please enter your 4-digit PIN: 1234
Login Successful! Current Balance: Rs. 50000
Enter withdrawal amount: 5000
Withdrawal: Rs. 5000.0
Service Charge (2%): Rs. 100.0
Remaining Balance: Rs. 44900.0

```

Question # 2:

Grocery Store Inventory System. Maintain item names, stock, and prices in a dictionary of dictionaries. User selects items with quantity. If stock available, deduct from inventory. Apply discount slabs (10%, 15%, 20%). Print final bill with tax (5%).

Code:

```
print("--- Welcome to Humna Imran's Grocery Store ---")
# 1. Inventory Setup
inventory_humna = {
    "Apple": {"price": 200, "stock": 50},
    "Bread": {"price": 150, "stock": 20},
    "Milk": {"price": 120, "stock": 30}
}
cart_total = 0
print("Available Items:", list(inventory_humna.keys()))# 2. Shopping Loop
while True:
    item = input("Enter item name (or 'done' to finish): ").capitalize()
    if item == 'Done':
        break

    if item in inventory_humna:
        qty = int(input(f"Enter quantity for {item}: "))

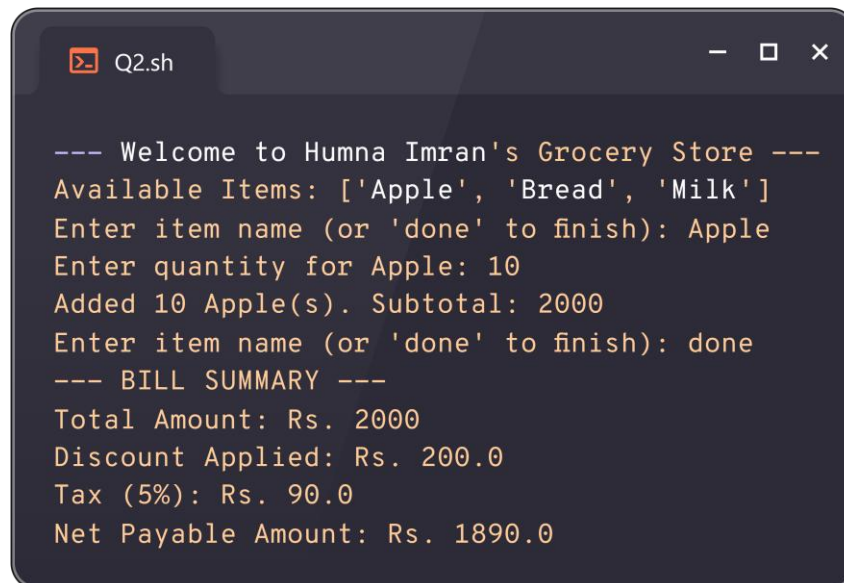
        if qty <= inventory_humna[item]["stock"]:
            # Update stock and calculate cost
            inventory_humna[item]["stock"] -= qty
            cost = qty * inventory_humna[item]["price"]
            cart_total += cost
            print(f"Added {qty} {item}(s). Subtotal: {cart_total}")
        else:
            print(f"Sorry, only {inventory_humna[item]['stock']} left in
stock.")
    else:
        print("Item not found!")
# 3. Apply Discount
discount = 0
if cart_total > 5000:
    discount = 0.20
elif cart_total > 3000:
    discount = 0.15
elif cart_total > 1000:
    discount = 0.10
discount_amount = cart_total * discount
tax_amount = (cart_total - discount_amount) * 0.05
final_bill_humna = (cart_total - discount_amount) + tax_amount
# 4. Final Bill
```

```

print("\n--- BILL SUMMARY ---")
print(f"Total Amount: Rs. {cart_total}")
print(f"Discount Applied: Rs. {discount_amount}")
print(f"Tax (5%): Rs. {tax_amount}")
print(f"Net Payable Amount: Rs. {final_bill_humna}")

```

output



```

Q2.sh
--- Welcome to Humna Imran's Grocery Store ---
Available Items: ['Apple', 'Bread', 'Milk']
Enter item name (or 'done' to finish): Apple
Enter quantity for Apple: 10
Added 10 Apple(s). Subtotal: 2000
Enter item name (or 'done' to finish): done
--- BILL SUMMARY ---
Total Amount: Rs. 2000
Discount Applied: Rs. 200.0
Tax (5%): Rs. 90.0
Net Payable Amount: Rs. 1890.0

```

Question # 3:

Student Grading with Subject Weightage. Input marks for 5 subjects where each subject has different weightage (stored in a tuple). Compute weighted average. Assign grade (A/B/C/D) based on the calculated percentage.

Code:

```

print("--- Humna Imran's Grade Calculator ---")

# 1. Define Weightages (Math, English, Science, Urdu, CS)
weights_humna = (0.3, 0.2, 0.2, 0.1, 0.2)
subjects = ["Math", "English", "Science", "Urdu", "CS"]
marks = []

# 2. Input Marks
for sub in subjects:
    m = float(input(f"Enter marks for {sub} (out of 100): "))
    marks.append(m)

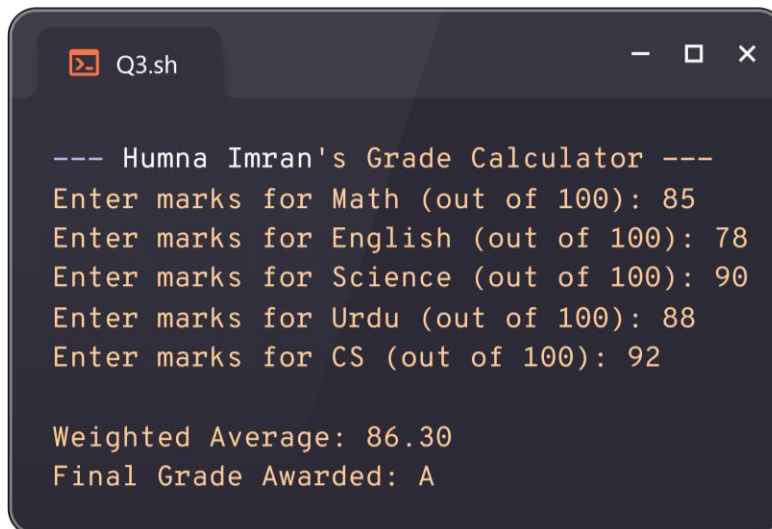
```

```

# 3. Compute Weighted Average
weighted_sum = 0
for i in range(len(marks)):
    weighted_sum += marks[i] * weights_humna[i]
print(f"\nWeighted Average: {weighted_sum:.2f}")
# 4. Assign Grade
if weighted_sum >= 90:
    grade = "A+"
elif weighted_sum >= 80:
    grade = "A"
elif weighted_sum >= 70:
    grade = "B"
elif weighted_sum >= 60:
    grade = "C"
else:
    grade = "F"
print(f"Final Grade Awarded: {grade}")

```

output



```

--- Humna Imran's Grade Calculator ---
Enter marks for Math (out of 100): 85
Enter marks for English (out of 100): 78
Enter marks for Science (out of 100): 90
Enter marks for Urdu (out of 100): 88
Enter marks for CS (out of 100): 92

Weighted Average: 86.30
Final Grade Awarded: A

```

Question # 4:

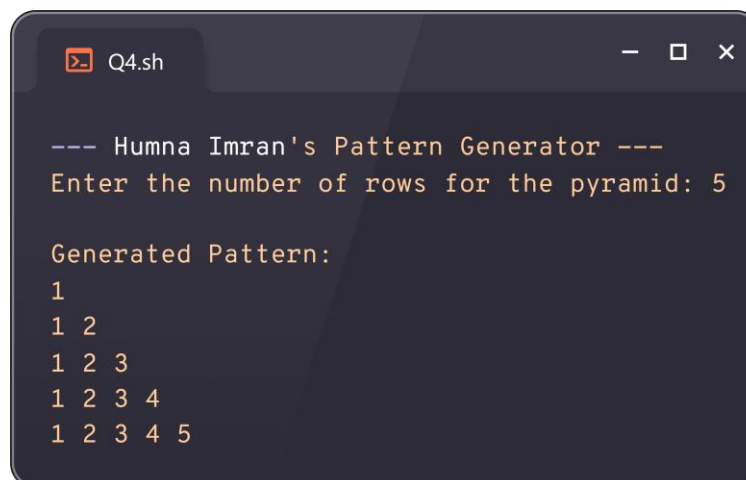
Generate a Number Pattern Pyramid. Write a program to take an integer input n from the user and generate a number pyramid where each row i contains numbers from 1 to i.

Code:

```
print("--- Humna Imran's Pattern Generator ---")

# 1. Get user input
rows_humna = int(input("Enter the number of rows for the pyramid: "))

# 2. Generate Pattern
print("\nGenerated Pattern:")
for i in range(1, rows_humna + 1):
    # Inner loop to print numbers
    for j in range(1, i + 1):
        print(j, end=" ")
    print() # Move to next line
```

output

```
Q4.sh

--- Humna Imran's Pattern Generator ---
Enter the number of rows for the pyramid: 5

Generated Pattern:
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Question # 5:

List Analysis and Filtering. Create a list of random numbers. Write a program to separate even and odd numbers into different lists, calculate the sum of each, and identify the largest number in the original list without using built-in max() function.

Code:

```
print("--- Humna Imran's List Analyzer ---")

# 1. Define the list
numbers_humna = [12, 45, 67, 23, 89, 34, 10, 56, 99, 4]
print(f"Original List: {numbers_humna}")
```



```

even_list = []
odd_list = []
max_val = numbers_humna[0]

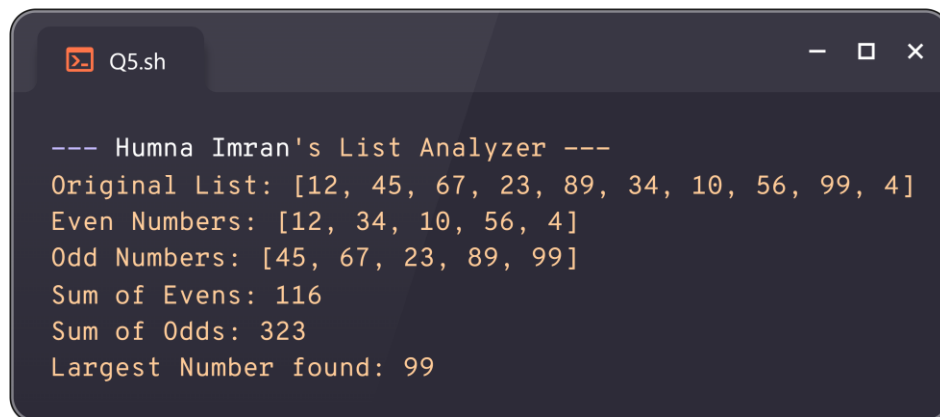
# 2. Process list
for num in numbers_humna:
    # Check even/odd
    if num % 2 == 0:
        even_list.append(num)
    else:
        odd_list.append(num)

# Find maximum manually
if num > max_val:
    max_val = num

# 3. Display Results
print(f"Even Numbers: {even_list}")
print(f"Odd Numbers: {odd_list}")
print(f"Sum of Evens: {sum(even_list)}")
print(f"Sum of Odds: {sum(odd_list)}")
print(f"Largest Number found: {max_val}")

```

output



```

Q5.sh
--- Humna Imran's List Analyzer ---
Original List: [12, 45, 67, 23, 89, 34, 10, 56, 99, 4]
Even Numbers: [12, 34, 10, 56, 4]
Odd Numbers: [45, 67, 23, 89, 99]
Sum of Evens: 116
Sum of Odds: 323
Largest Number found: 99

```

LAB # 2

Advanced Logic & System Simulations

Question # 6:

Recursive Sequence Generator. Write a program that asks the user for a number n. Implement two functions: one to calculate the factorial of n using recursion, and another to generate the Fibonacci sequence up to the n-th term. Display both results.

Code:

```
print("--- Humna Imran's Math Logic System ---")

# 1. Recursive Factorial Function
def factorial_humna(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial_humna(n - 1)

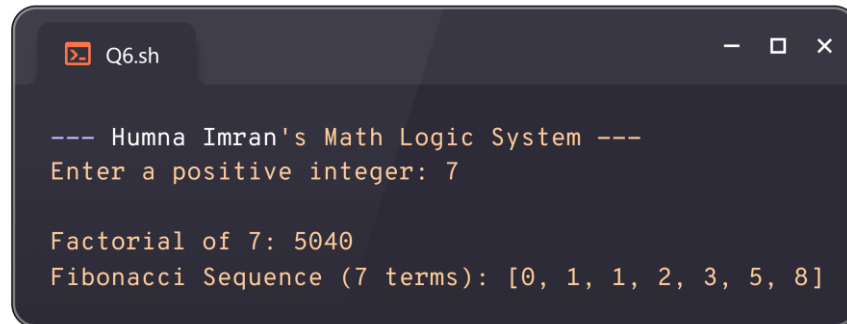
# 2. Fibonacci Sequence Function
def fibonacci_humna(n):
    fib_sequence = [0, 1]
    for i in range(2, n):
        next_val = fib_sequence[-1] + fib_sequence[-2]
        fib_sequence.append(next_val)
    return fib_sequence[:n]

# 3. Main Logic
num_input = int(input("Enter a positive integer: "))

if num_input < 0:
    print("Please enter a positive number.")
else:
    fact_result = factorial_humna(num_input)
    fib_result = fibonacci_humna(num_input)

    print(f"\nFactorial of {num_input}: {fact_result}")
    print(f"Fibonacci Sequence ({num_input} terms): {fib_result}")
```

output



```
Q6.sh

--- Humna Imran's Math Logic System ---
Enter a positive integer: 7

Factorial of 7: 5040
Fibonacci Sequence (7 terms): [0, 1, 1, 2, 3, 5, 8]
```

Question # 7:

Password Strength Validator. Simulate a security system that checks a user's password strength. The password must be at least 8 characters long, contain numbers, uppercase letters, and special characters (@, #, \$). Provide feedback on what is missing.

Code:

```
print("--- Humna Imran's Security System ---")

def check_password_strength(password):
    strength_score = 0
    feedback_humna = []

    # 1. Check Length
    if len(password) >= 8:
        strength_score += 1
    else:
        feedback_humna.append("Too short (needs 8+ chars)")

    # 2. Check for Numbers
    if any(char.isdigit() for char in password):
        strength_score += 1
    else:
        feedback_humna.append("Missing numbers")

    # 3. Check for Uppercase
    if any(char.isupper() for char in password):
        strength_score += 1
    else:
        feedback_humna.append("Missing uppercase letters")
```

```

# 4. Check for Special Characters
special_chars = "@#$!%^&*"
if any(char in special_chars for char in password):
    strength_score += 1
else:
    feedback_humna.append("Missing special characters")

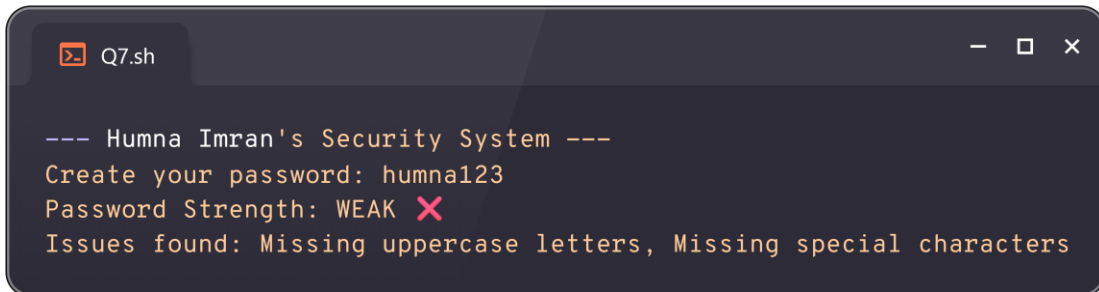
return strength_score, feedback_humna

# Main execution
user_pass = input("Create your password: ")
score, issues = check_password_strength(user_pass)

if score == 4:
    print("Password Strength: STRONG ☑")
else:
    print("Password Strength: WEAK ✖")
    print("Issues found:", ", ".join(issues))

```

output



```

Q7.sh
--- Humna Imran's Security System ---
Create your password: humna123
Password Strength: WEAK ✖
Issues found: Missing uppercase letters, Missing special characters

```

Question # 8:

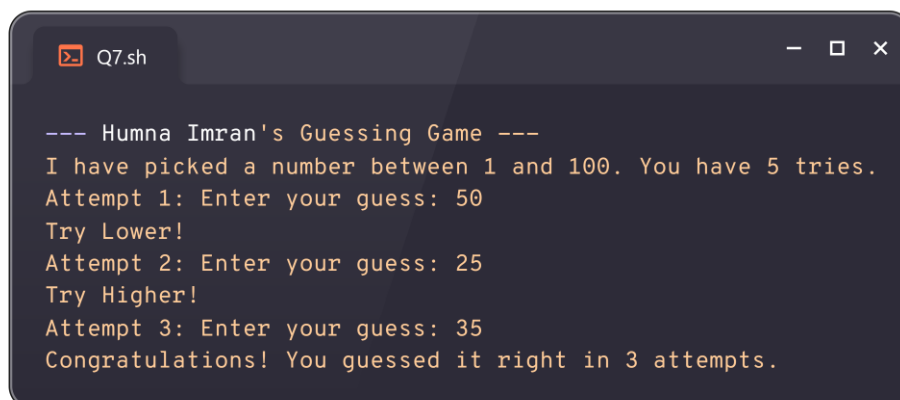
Number Guessing Game Simulation. The system generates a random number between 1 and 100. The user has 5 attempts to guess it. The system provides "Higher" or "Lower" hints. If the user fails, reveal the number.

Code:

```
import random
print("--- Humna Imran's Guessing Game ---")
# 1. Setup Game
target_number = random.randint(1, 100)
attempts_humna = 5
print(f"I have picked a number between 1 and 100. You have {attempts_humna} tries.")
# 2. Game Loop
for i in range(1, attempts_humna + 1):
    guess = int(input(f"Attempt {i}: Enter your guess: "))

    if guess == target_number:
        print(f"Congratulations! You guessed it right in {i} attempts.")
        break
    elif guess < target_number:
        print("Try Higher!")
    else:
        print("Try Lower!")
else:
    # This executes if loop finishes without break
    print(f"\nGame Over! The correct number was {target_number}.")
```

output

A terminal window titled 'Q7.sh' with standard window controls (minimize, maximize, close). The output text is as follows:

```
--- Humna Imran's Guessing Game ---
I have picked a number between 1 and 100. You have 5 tries.
Attempt 1: Enter your guess: 50
Try Lower!
Attempt 2: Enter your guess: 25
Try Higher!
Attempt 3: Enter your guess: 35
Congratulations! You guessed it right in 3 attempts.
```

Question # 9:

File-Based Student Logger. Create a program that accepts student names and their marks, then writes this data into a text file named student_log_humna.txt. After writing, read the file back and display the content to verify the data storage.

Code:

```
print("--- Humna Imran's Data Logger ---")

filename = "student_log_humna.txt"

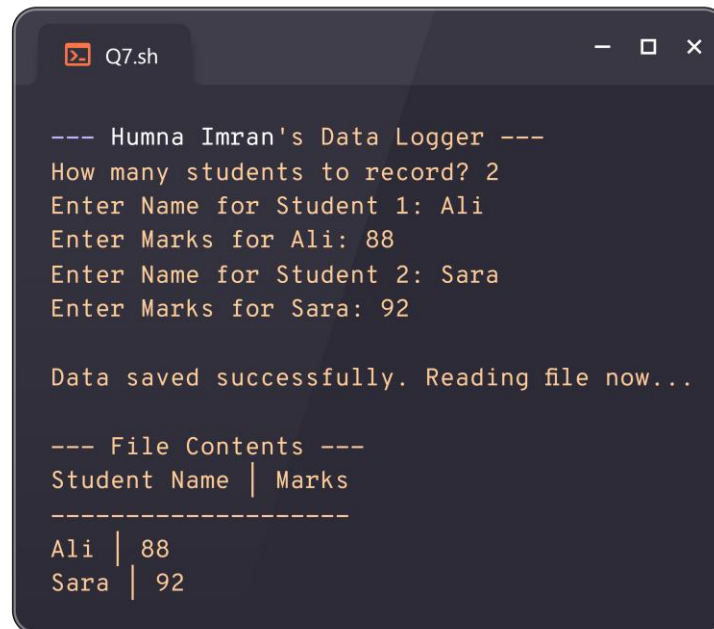
# 1. Write Data to File
num_students = int(input("How many students to record? "))

with open(filename, "w") as file:
    file.write("Student Name | Marks\n")
    file.write("-----\n")
    for i in range(num_students):
        name = input(f"Enter Name for Student {i+1}: ")
        marks = input(f"Enter Marks for {name}: ")
        file.write(f"{name} | {marks}\n")

print("\nData saved successfully. Reading file now...\n")

# 2. Read Data from File
try:
    with open(filename, "r") as file:
        content_humna = file.read()
        print("--- File Contents ---")
        print(content_humna)
except FileNotFoundError:
    print("Error: File not found.")
```

output



```
Q7.sh

--- Humna Imran's Data Logger ---
How many students to record? 2
Enter Name for Student 1: Ali
Enter Marks for Ali: 88
Enter Name for Student 2: Sara
Enter Marks for Sara: 92

Data saved successfully. Reading file now...

--- File Contents ---
Student Name | Marks
-----
Ali | 88
Sara | 92
```

Question # 10:

Smart Thermostat Simulation using Classes. Create a class SmartThermostat with an initial temperature. Include methods to increase_temp, decrease_temp, and reset_temp. Ensure the temperature does not exceed 30°C or drop below 16°C.

Code:

```
print("--- Humna Imran's Smart Home System ---")

class SmartThermostat:
    def __init__(self, initial_temp):
        self.temp_humna = initial_temp
        print(f"Thermostat started at {self.temp_humna}°C")

    def increase_temp(self):
        if self.temp_humna < 30:
            self.temp_humna += 1
            print(f"Temperature increased to {self.temp_humna}°C")
        else:
            print("Max temperature (30°C) reached!")

    def decrease_temp(self):
        if self.temp_humna > 16:
```

```

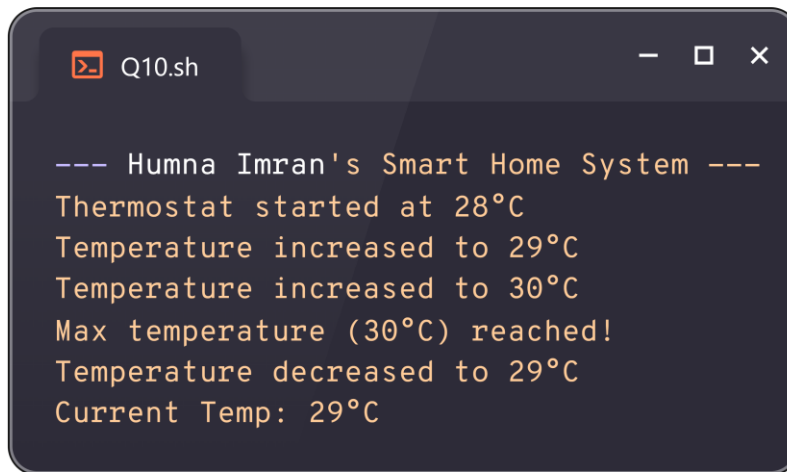
        self.temp_humna -= 1
        print(f"Temperature decreased to {self.temp_humna}°C")
    else:
        print("Min temperature (16°C) reached!")

    def get_status(self):
        return f"Current Temp: {self.temp_humna}°C"

# Simulation
device = SmartThermostat(28)
device.increase_temp()
device.increase_temp() # Should hit max
device.increase_temp() # Should warn
device.decrease_temp()
print(device.get_status())

```

output



```

Q10.sh

--- Humna Imran's Smart Home System ---
Thermostat started at 28°C
Temperature increased to 29°C
Temperature increased to 30°C
Max temperature (30°C) reached!
Temperature decreased to 29°C
Current Temp: 29°C

```


LAB # 3

Data Handling with Dictionaries & Tuples

Question # 11:

Contact Management System. Create a program to manage a phonebook using a dictionary. The user should be able to add a new contact, search for an existing contact, and delete a contact. Ensure that duplicate names are handled (i.e., prevent overwriting without warning).

Code:

```
print("--- Humna Imran's Contact Manager ---")

contacts_humna = {}

while True:
    print("\n1. Add Contact  2. Search Contact  3. Delete Contact  4. Exit")
    choice = input("Enter your choice (1-4): ")

    if choice == '1':
        name = input("Enter Name: ").title()
        if name in contacts_humna:
            print("Contact already exists!")
        else:
            phone = input("Enter Phone Number: ")
            contacts_humna[name] = phone
            print(f"Contact {name} added successfully.")

    elif choice == '2':
        name = input("Enter Name to Search: ").title()
        # Using .get() to handle missing keys gracefully
        number = contacts_humna.get(name, "Not Found")
        print(f"Result: {name} - {number}")

    elif choice == '3':
        name = input("Enter Name to Delete: ").title()
        if name in contacts_humna:
            del contacts_humna[name]
            print(f"Contact {name} deleted.")
        else:
```

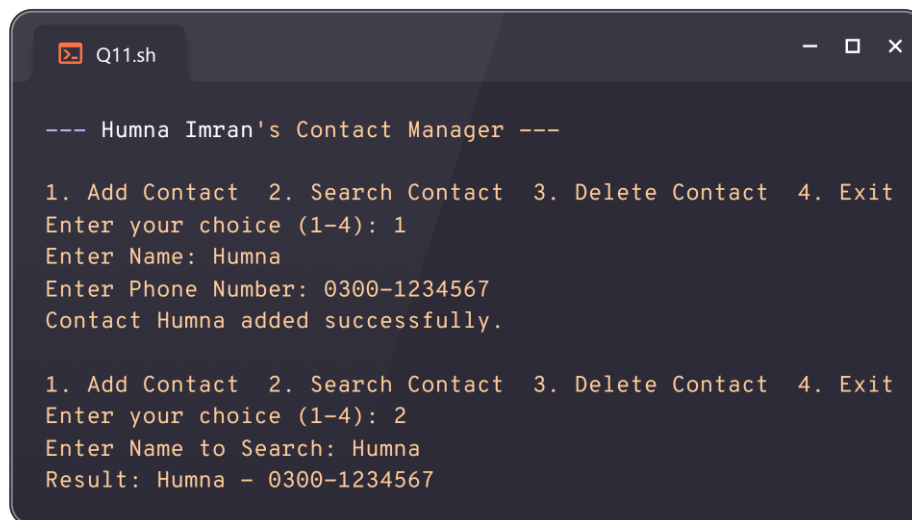
```

        print("Contact not found.")

    elif choice == '4':
        print("Exiting Contact Manager...")
        break
    else:
        print("Invalid Option. Please try again.")

```

output



```

Q11.sh
--- Humna Imran's Contact Manager ---

1. Add Contact  2. Search Contact  3. Delete Contact  4. Exit
Enter your choice (1-4): 1
Enter Name: Humna
Enter Phone Number: 0300-1234567
Contact Humna added successfully.

1. Add Contact  2. Search Contact  3. Delete Contact  4. Exit
Enter your choice (1-4): 2
Enter Name to Search: Humna
Result: Humna - 0300-1234567

```

Question # 12:

Word Frequency Counter. Write a program that takes a sentence input from the user and counts the frequency of each word using a dictionary. The program should be case-insensitive and ignore punctuation.

Code:

```

import string

print("--- Humna Imran's Text Analyzer ---")

# 1. Input Processing
text_input = input("Enter a sentence to analyze: ").lower()

# Remove punctuation
for char in string.punctuation:

```

```

        text_input = text_input.replace(char, "")

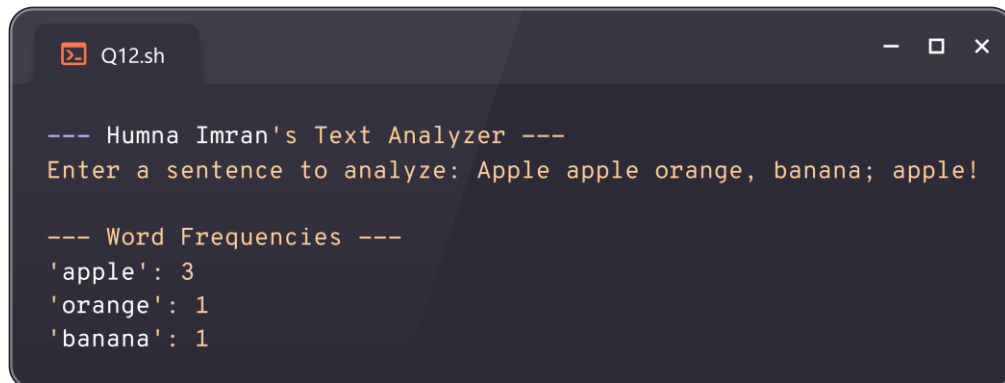
words_list = text_input.split()
frequency_humna = {}

# 2. Count Frequencies
for word in words_list:
    if word in frequency_humna:
        frequency_humna[word] += 1
    else:
        frequency_humna[word] = 1

# 3. Display Results
print("\n--- Word Frequencies ---")
for word, count in frequency_humna.items():
    print(f"'{word}': {count}")

```

output



```

Q12.sh
--- Humna Imran's Text Analyzer ---
Enter a sentence to analyze: Apple apple orange, banana; apple!

--- Word Frequencies ---
'apple': 3
'orange': 1
'banana': 1

```

Question # 13:

Student Record Processing with Tuples. Create a list of tuples where each tuple contains (Student Name, Age, Grade). Write a program to:

- Print the details of all students.
- Filter and display students who have a Grade of 'A'.
- Find the average age of the students.

Code:

```

print("--- Humna Imran's Student Records ---")

# List of Tuples: (Name, Age, Grade)

```

```

students_humna = [
    ("Ali", 20, "A"),
    ("Sara", 21, "B"),
    ("Ahmed", 19, "A"),
    ("Zain", 22, "C"),
    ("Hira", 20, "A")
]

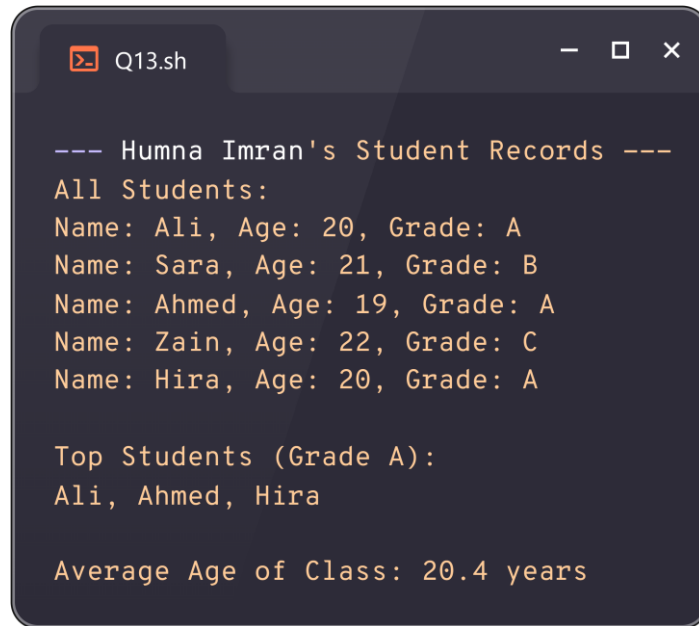
# 1. Display All
print("All Students:")
for record in students_humna:
    print(f"Name: {record[0]}, Age: {record[1]}, Grade: {record[2]}")

# 2. Filter 'A' Graders
print("\nTop Students (Grade A):")
a_graders = [s[0] for s in students_humna if s[2] == "A"]
print(", ".join(a_graders))

# 3. Average Age
total_age = sum(s[1] for s in students_humna)
avg_age = total_age / len(students_humna)
print(f"\nAverage Age of Class: {avg_age:.1f} years")

```

output

A terminal window titled 'Q13.sh' with standard window controls (minimize, maximize, close). The output text is as follows:

```
--- Humna Imran's Student Records ---  
All Students:  
Name: Ali, Age: 20, Grade: A  
Name: Sara, Age: 21, Grade: B  
Name: Ahmed, Age: 19, Grade: A  
Name: Zain, Age: 22, Grade: C  
Name: Hira, Age: 20, Grade: A  
  
Top Students (Grade A):  
Ali, Ahmed, Hira  
  
Average Age of Class: 20.4 years
```

Question # 14:

Inventory Merger. You have two dictionaries representing the inventory of two different warehouses. Write a script to merge them into a single dictionary. If an item exists in both, sum up their quantities.

Code:

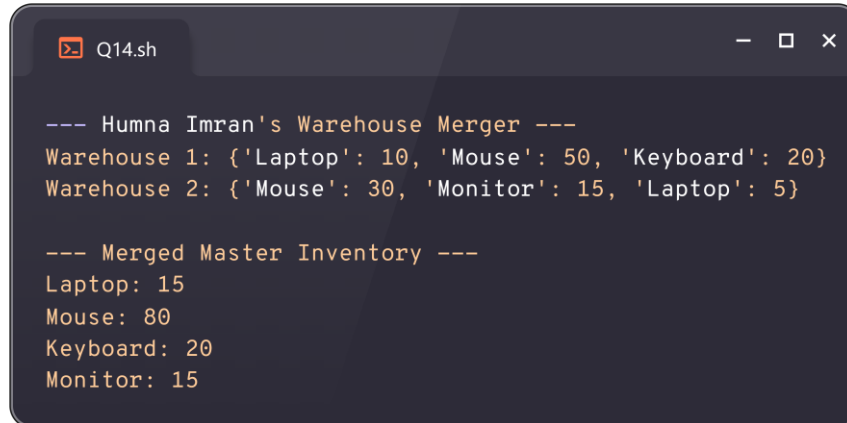
```
print("--- Humna Imran's Warehouse Merger ---")  
  
warehouse_1 = {'Laptop': 10, 'Mouse': 50, 'Keyboard': 20}  
warehouse_2 = {'Mouse': 30, 'Monitor': 15, 'Laptop': 5}  
  
# Copy first warehouse data to the master inventory  
master_inventory_humna = warehouse_1.copy()  
  
# Merge second warehouse data  
for item, qty in warehouse_2.items():  
    if item in master_inventory_humna:  
        # If item exists, add to existing stock  
        master_inventory_humna[item] += qty  
    else:  
        # If new item, add to inventory  
        master_inventory_humna[item] = qty
```

```

print("Warehouse 1:", warehouse_1)
print("Warehouse 2:", warehouse_2)
print("\n--- Merged Master Inventory ---")
for item, qty in master_inventory_humna.items():
    print(f"{item}: {qty}")

```

output



```

Q14.sh

--- Humna Imran's Warehouse Merger ---
Warehouse 1: {'Laptop': 10, 'Mouse': 50, 'Keyboard': 20}
Warehouse 2: {'Mouse': 30, 'Monitor': 15, 'Laptop': 5}

--- Merged Master Inventory ---
Laptop: 15
Mouse: 80
Keyboard: 20
Monitor: 15

```

Question # 15:

Dictionary Inversion. Write a program to invert a dictionary mapping ID -> Name to Name -> ID. Ensure that if multiple IDs map to the same name, the new dictionary stores the IDs in a list under that name.

Code:

```

print("--- Humna Imran's ID Mapper ---")

id_to_name = {
    101: "Humna",
    102: "Ali",
    103: "Sara",
    104: "Humna", # Duplicate name
    105: "Zain"
}

name_to_id_humna = {}

for emp_id, name in id_to_name.items():
    if name in name_to_id_humna:
        # If name exists, append ID to list

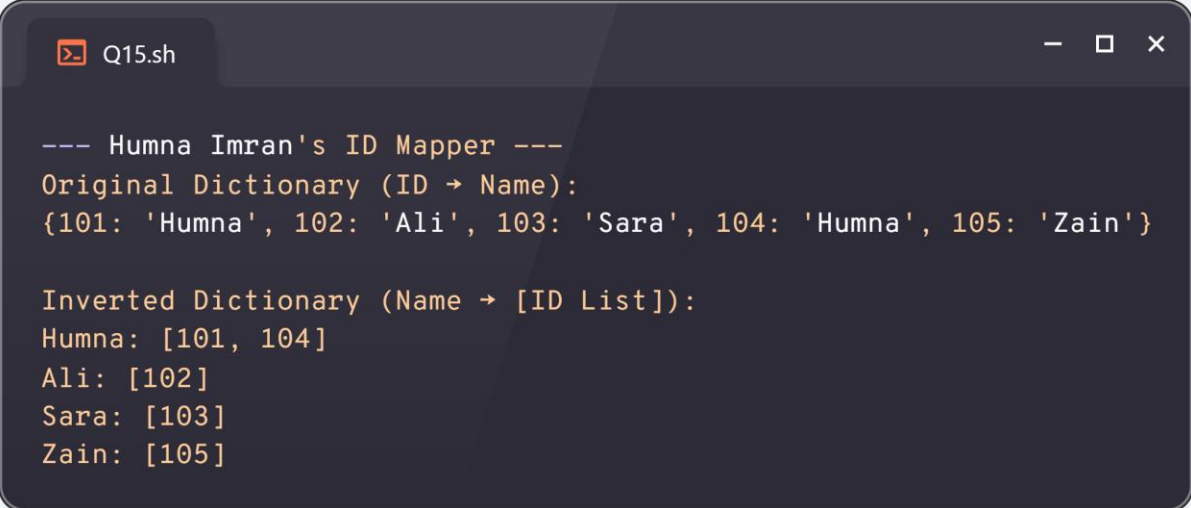
```

```
        name_to_id_humna[name].append(emp_id)
    else:
        # Create new list with ID
        name_to_id_humna[name] = [emp_id]

print("Original Dictionary (ID -> Name):")
print(id_to_name)

print("\nInverted Dictionary (Name -> [ID List]):")
for name, ids in name_to_id_humna.items():
    print(f"{name}: {ids}")
```

output



```
Q15.sh

--- Humna Imran's ID Mapper ---
Original Dictionary (ID -> Name):
{101: 'Humna', 102: 'Ali', 103: 'Sara', 104: 'Humna', 105: 'Zain'}

Inverted Dictionary (Name -> [ID List]):
Humna: [101, 104]
Ali: [102]
Sara: [103]
Zain: [105]
```

LAB # 4

Functional Programming & List Processing

Question # 16:

Lambda Sorting. You have a list of tuples representing products and their prices (Product, Price). Write a program using a lambda function to sort this list in descending order based on the price.

Code:

```
print("--- Humna Imran's Price Sorter ---")

# List of tuples (Item, Price)
products_humna = [
    ("Laptop", 120000),
    ("Mouse", 1500),
    ("Keyboard", 3000),
    ("Monitor", 25000),
    ("USB", 800)
]

print("Original List:", products_humna)

# 1. Sort using lambda (Descending order of price)
sorted_products = sorted(products_humna, key=lambda x: x[1],
reverse=True)

# 2. Display Results
print("\nSorted by Price (High to Low):")
for item in sorted_products:
    print(f"{item[0]}: Rs. {item[1]}")
```


output

```
Q16.sh
--- Humna Imran's Price Sorter ---
Original List: [('Laptop', 120000), ('Mouse', 1500), ('Keyboard', 3000), ('Monitor', 25000), ('USB', 800)]

Sorted by Price (High to Low):
Laptop: Rs. 120000
Monitor: Rs. 25000
Keyboard: Rs. 3000
Mouse: Rs. 1500
USB: Rs. 800
```

Question # 17:

Map, Filter, and Reduce. Create a list of integers. Use filter() to extract odd numbers, map() to square them, and reduce() (from functools) to calculate the sum of these squared numbers.

Code:

```
from functools import reduce

print("--- Humna Imran's Functional Pipeline ---")

numbers_humna = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(f"Original Numbers: {numbers_humna}")

# 1. Filter Odd Numbers
odd_nums = list(filter(lambda x: x % 2 != 0, numbers_humna))
print(f"Filtered Odds: {odd_nums}")

# 2. Map to Squares
squared_odds = list(map(lambda x: x ** 2, odd_nums))
print(f"Squared Odds: {squared_odds}")

# 3. Reduce to Sum
total_sum = reduce(lambda x, y: x + y, squared_odds)
print(f"Sum of Squares: {total_sum}")
```

output

```
Q16.sh
--- Humna Imran's Functional Pipeline ---
Original Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Filtered Odds: [1, 3, 5, 7, 9]
Squared Odds: [1, 9, 25, 49, 81]
Sum of Squares: 165
```

Question # 18:

Advanced List Comprehension. Write a program that takes a sentence and uses list comprehension to create a list of words that are longer than 3 characters and converts them to uppercase.

Code:

```
print("--- Humna Imran's Text Processor ---")

sentence = "The quick brown fox jumps over the lazy dog"
print(f"Input Sentence: '{sentence}'")

# 1. List Comprehension
# Logic: uppercase word FOR word IN split_sentence IF length > 3
processed_words_humna = [word.upper() for word in sentence.split() if
len(word) > 3]

# 2. Display Result
print("\nProcessed Words (> 3 chars & Uppercase):")
print(processed_words_humna)
```

output

```
Q16.sh
--- Humna Imran's Text Processor ---
Input Sentence: 'The quick brown fox jumps over the lazy dog'

Processed Words (> 3 chars & Uppercase):
['QUICK', 'BROWN', 'JUMPS', 'OVER', 'LAZY']
```

Question # 19:

Custom Generator. Create a generator function `square_generator(limit)` that yields the square of numbers from 1 up to the limit. Use a loop to iterate through the generator and print the values.

Code:

```
print("--- Humna Imran's Number Generator ---")

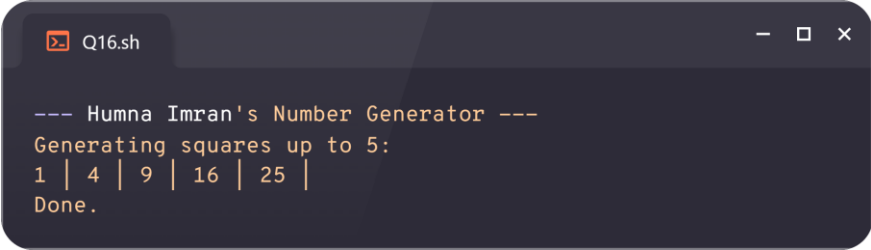
# 1. Define Generator Function
def square_generator_humna(limit):
    n = 1
    while n <= limit:
        yield n * n # Return value and pause state
        n += 1

# 2. Use the Generator
limit_input = 5
print(f"Generating squares up to {limit_input}:")

gen = square_generator_humna(limit_input)

for value in gen:
    print(value, end=" | ")
print("\nDone.")
```

output



```
Q16.sh
--- Humna Imran's Number Generator ---
Generating squares up to 5:
1 | 4 | 9 | 16 | 25 |
Done.
```

Question # 20:

Decorator for Logging. Write a decorator named `execution_logger` that prints "Starting execution..." before a function runs and "Execution finished." after it runs. Apply this decorator to a function that calculates the cube of a number.

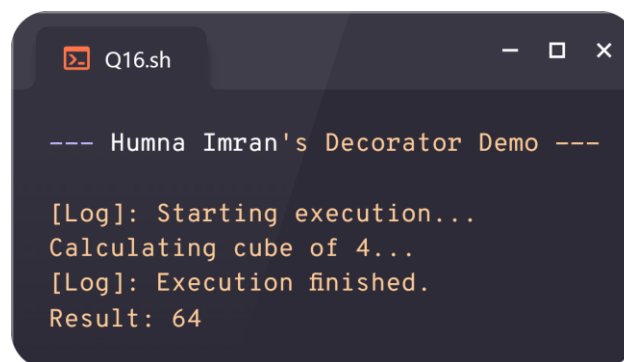
Code:

```
print("--- Humna Imran's Decorator Demo ---")

# 1. Define Decorator
def execution_logger(func):
    def wrapper(x):
        print("\n[Log]: Starting execution...")
        result = func(x)
        print("[Log]: Execution finished.")
        return result
    return wrapper

# 2. Apply Decorator
@execution_logger
def calculate_cube_humna(num):
    print(f"Calculating cube of {num}...")
    return num ** 3

# 3. Test Function
value = 4
cube_res = calculate_cube_humna(value)
print(f"Result: {cube_res}")
```

outputA terminal window titled 'Q16.sh' with standard window controls. It displays the output of the Python script: a separator line, a log message for starting execution, the calculation of the cube of 4, a log message for execution finished, and the final result of 64.

```
Q16.sh

--- Humna Imran's Decorator Demo ---

[Log]: Starting execution...
Calculating cube of 4...
[Log]: Execution finished.
Result: 64
```

LAB # 5

NumPy - Array Creation & Indexing

Question # 21:

Basic Array Creation. Write a program using NumPy to create:

1. A 1D array of numbers from 0 to 9.
2. A 3x3 matrix of all ones.
3. A 2x2 matrix of all zeros.

Print the shape and data type of the 3x3 matrix.

Code:

```
import numpy as np
print("--- Humna Imran's NumPy Initialization ---")

# 1. 1D Array
arr_1d_humna = np.arange(10)
print("1D Array (0-9):", arr_1d_humna)

# 2. 3x3 Matrix of Ones
ones_matrix = np.ones((3, 3))
print("\n3x3 Matrix of Ones:\n", ones_matrix)

# 3. 2x2 Matrix of Zeros
zeros_matrix = np.zeros((2, 2))
print("\n2x2 Matrix of Zeros:\n", zeros_matrix)

# 4. Properties
print("\nProperties of Ones Matrix:")
print(f"Shape: {ones_matrix.shape}")
print(f>Data Type: {ones_matrix.dtype}")
```

output

```
Q16.sh

--- Humna Imran's NumPy Initialization ---
1D Array (0-9): [0 1 2 3 4 5 6 7 8 9]

3x3 Matrix of Ones:
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]

2x2 Matrix of Zeros:
[[0. 0.]
 [0. 0.]]

Properties of Ones Matrix:
Shape: (3, 3)
Data Type: float64
```

Question # 22:

Array Reshaping. Create a 1D array containing numbers from 1 to 12. Reshape this array into a 3x4 matrix and then into a 4x3 matrix. Finally, verify that the total number of elements remains unchanged.

Code:

```
import numpy as np
print("--- Humna Imran's Shape Shifter ---")

# 1. Create Initial Array
flat_arr = np.arange(1, 13)
print(f"Original 1D Array: {flat_arr}")

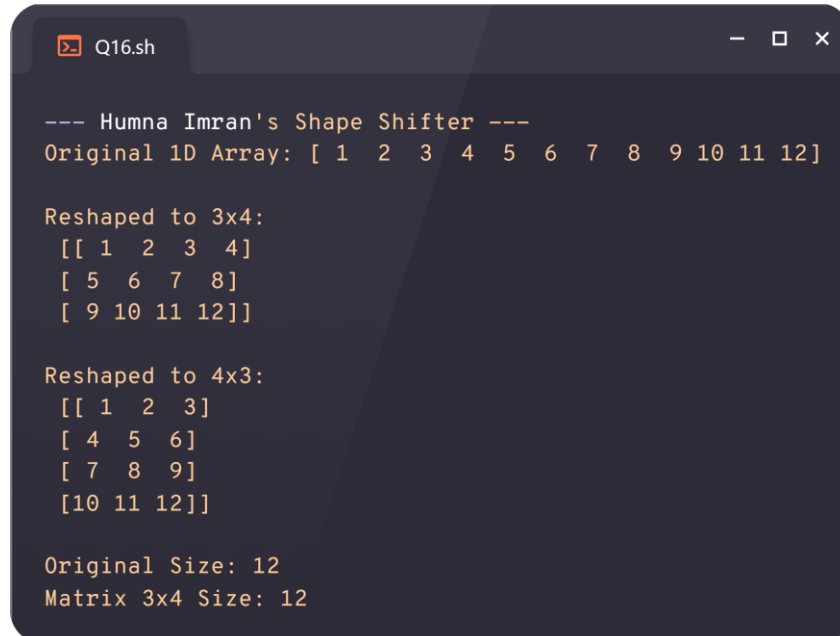
# 2. Reshape to 3x4
matrix_3x4_humna = flat_arr.reshape(3, 4)
print("\nReshaped to 3x4:\n", matrix_3x4_humna)

# 3. Reshape to 4x3
matrix_4x3_humna = flat_arr.reshape(4, 3)
print("\nReshaped to 4x3:\n", matrix_4x3_humna)

# 4. Verification
```

```
print(f"\nOriginal Size: {flat_arr.size}")
print(f"Matrix 3x4 Size: {matrix_3x4_humna.size}")
```

output



```
Q16.sh

--- Humna Imran's Shape Shifter ---
Original 1D Array: [ 1  2  3  4  5  6  7  8  9 10 11 12]

Reshaped to 3x4:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

Reshaped to 4x3:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

Original Size: 12
Matrix 3x4 Size: 12
```

Question # 23:

Slicing and Indexing. Create a 5x5 identity matrix multiplied by 5 (so the diagonal is 5). Extract and print:

1. The first row.
2. The last column.
3. The central 3x3 sub-matrix.

Code:

```
import numpy as np
print("--- Humna Imran's Matrix Slicer ---")

# 1. Create Matrix (Identity * 5)
matrix_humna = np.eye(5) * 5
print("Original 5x5 Matrix:\n", matrix_humna)

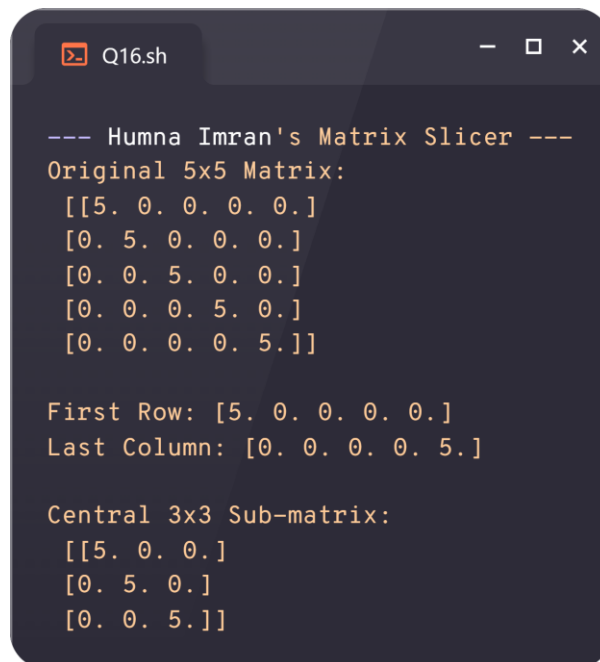
# 2. Extract First Row
print("\nFirst Row:", matrix_humna[0])

# 3. Extract Last Column
```

```
# Using negative indexing for last column
print("Last Column:", matrix_humna[:, -1])

# 4. Central 3x3 Sub-matrix
# Rows 1 to 3 (exclusive of 4), Cols 1 to 3 (exclusive of 4)
sub_matrix = matrix_humna[1:4, 1:4]
print("\nCentral 3x3 Sub-matrix:\n", sub_matrix)
```

output



```
Q16.sh

--- Humna Imran's Matrix Slicer ---
Original 5x5 Matrix:
[[5. 0. 0. 0. 0.]
 [0. 5. 0. 0. 0.]
 [0. 0. 5. 0. 0.]
 [0. 0. 0. 5. 0.]
 [0. 0. 0. 0. 5.]]

First Row: [5. 0. 0. 0. 0.]
Last Column: [0. 0. 0. 0. 5.]

Central 3x3 Sub-matrix:
[[5. 0. 0.]
 [0. 5. 0.]
 [0. 0. 5.]]
```

Question # 24:

Linspace and Random Numbers. Use linspace to generate 5 equally spaced numbers between 0 and 10. Then, generate an array of 5 random integers between 10 and 50. Add these two arrays together (element-wise) and print the result.

Code:

```
import numpy as np
print("--- Humna Imran's Random Generator ---")

# 1. Linspace (5 numbers between 0 and 10)
spaced_arr = np.linspace(0, 10, 5)
print("Linspace Array:", spaced_arr)

# 2. Random Integers
# Seed for reproducibility
```



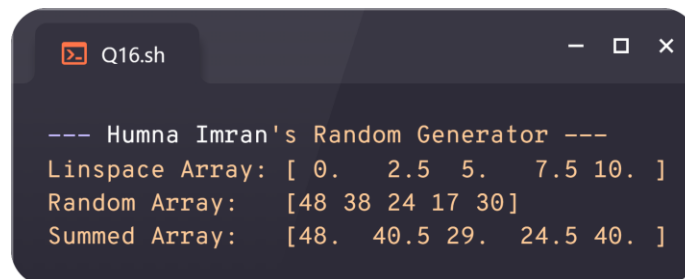
```

np.random.seed(42)
random_arr_humna = np.random.randint(10, 50, 5)
print("Random Array: ", random_arr_humna)

# 3. Element-wise Addition
result_arr = spaced_arr + random_arr_humna
print("Summed Array: ", result_arr)

```

output



```

Q16.sh
--- Humna Imran's Random Generator ---
Linspace Array: [ 0.  2.5  5.  7.5 10. ]
Random Array:   [48 38 24 17 30]
Summed Array:   [48.  40.5 29.  24.5 40. ]

```

Question # 25:

Boolean Indexing (Filtering). Create an array of 10 random integers between 1 and 100. Write a program to identify and print all numbers in the array that are greater than 50. Also replace all numbers less than 30 with -1.

Code:

```

import numpy as np
print("--- Humna Imran's Data Filter ---")

np.random.seed(10)
data_arr_humna = np.random.randint(1, 100, 10)
print("Original Data:", data_arr_humna)

# 1. Filter > 50
high_values = data_arr_humna[data_arr_humna > 50]
print("Values > 50: ", high_values)

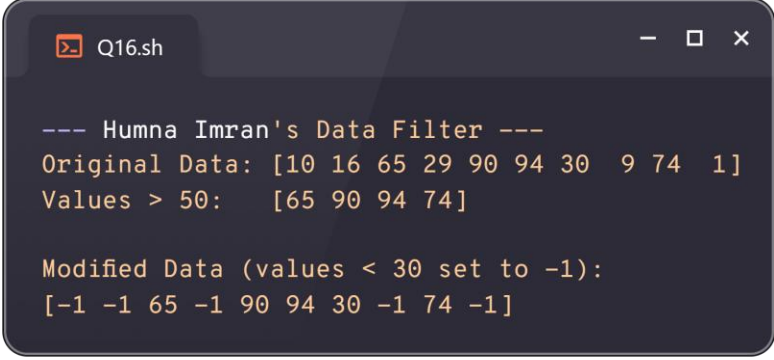
# 2. Replace < 30 with -1
# Using boolean masking for assignment
data_arr_humna[data_arr_humna < 30] = -1

print("\nModified Data (values < 30 set to -1):")

```

```
print(data_arr_humna)
```

output

A terminal window titled 'Q16.sh' with standard window controls. It displays the output of a script. The output includes a separator line, the original data array, a filtered array of values greater than 50, another separator line, a description of the modification, and the final modified data array.

```
Q16.sh

--- Humna Imran's Data Filter ---
Original Data: [10 16 65 29 90 94 30  9 74  1]
Values > 50:   [65 90 94 74]

---
Modified Data (values < 30 set to -1):
[-1 -1 65 -1 90 94 30 -1 74 -1]
```

LAB # 8

Pandas - Data Cleaning & Transformation

Question # 36:

Handling Missing Data. Create a DataFrame with missing values (NaN). Write a program to:

1. Identify missing values.
2. Fill missing numerical values with the mean of the column.
3. Drop rows where any remaining data is missing.

Code:

```
import pandas as pd
import numpy as np
print("--- Humna Imran's Data Cleaner ---")

# 1. Create DataFrame with Missing Data
data = {
    'Product': ['A', 'B', 'C', 'D', 'E'],
    'Sales': [100, np.nan, 150, 200, np.nan],
    'Cost': [80, 50, np.nan, 120, 90]
}
df_humna = pd.DataFrame(data)
print("Original Data:\n", df_humna)

# 2. Check for Missing Values
print("\nMissing Values Count:\n", df_humna.isnull().sum())

# 3. Fill Missing Sales with Mean
mean_sales = df_humna['Sales'].mean()
df_humna['Sales'].fillna(mean_sales, inplace=True)
print(f"\nData after filling Sales (Mean: {mean_sales}):\n", df_humna)

# 4. Drop Rows with any NaNs (Cost column still has NaNs)
df_clean = df_humna.dropna()
print("\nData after dropping rows with NaNs:\n", df_clean)
```

output

```
Q16.sh

--- Humna Imran's Data Cleaner ---
Original Data:
  Product  Sales  Cost
0      A  100.0  80.0
1      B   NaN  50.0
2      C  150.0   NaN
3      D  200.0 120.0
4      E   NaN  90.0

Missing Values Count:
  Product    0
  Sales      2
  Cost       1
dtype: int64

Data after filling Sales (Mean: 150.0):
  Product  Sales  Cost
0      A  100.0  80.0
1      B  150.0  50.0
2      C  150.0   NaN
3      D  200.0 120.0
4      E  150.0  90.0

Data after dropping rows with NaNs:
  Product  Sales  Cost
0      A  100.0  80.0
1      B  150.0  50.0
3      D  200.0 120.0
4      E  150.0  90.0
```

Question # 37:

Grouping and Aggregation. Given a sales dataset with columns Region, Product, and Revenue, group the data by Region to calculate the total revenue per region. Also, find the average revenue per product.

Code:

```
import pandas as pd
print("--- Humna Imran's Sales Aggregator ---")
```

```

data = {
    'Region': ['North', 'South', 'North', 'East', 'South', 'East'],
    'Product': ['A', 'A', 'B', 'A', 'B', 'B'],
    'Revenue': [1000, 1500, 1200, 1100, 1600, 1300]
}
df_sales_humna = pd.DataFrame(data)

# 1. Group by Region (Total Revenue)
region_groups = df_sales_humna.groupby('Region')['Revenue'].sum()
print("Total Revenue by Region:\n", region_groups)

# 2. Group by Product (Average Revenue)
product_avg = df_sales_humna.groupby('Product')['Revenue'].mean()
print("\nAverage Revenue by Product:\n", product_avg)

```

output

```

Q16.sh

--- Humna Imran's Sales Aggregator ---
Total Revenue by Region:
  Region
East    2400
North   2200
South   3100
Name: Revenue, dtype: int64

Average Revenue by Product:
  Product
A      1200.0
B    1366.666667
Name: Revenue, dtype: float64

```

Question # 38:

Merging DataFrames. You have two DataFrames: Customers (ID, Name) and Orders (OrderID, ID, Amount). Perform an Inner Join to combine them and display the purchase history of customers.

Code:

```

import pandas as pd
print("--- Humna Imran's Data Merger ---")

```

```

# 1. Create DataFrames
customers = pd.DataFrame({
    'ID': [101, 102, 103],
    'Name': ['Ali', 'Sara', 'Zain']
})

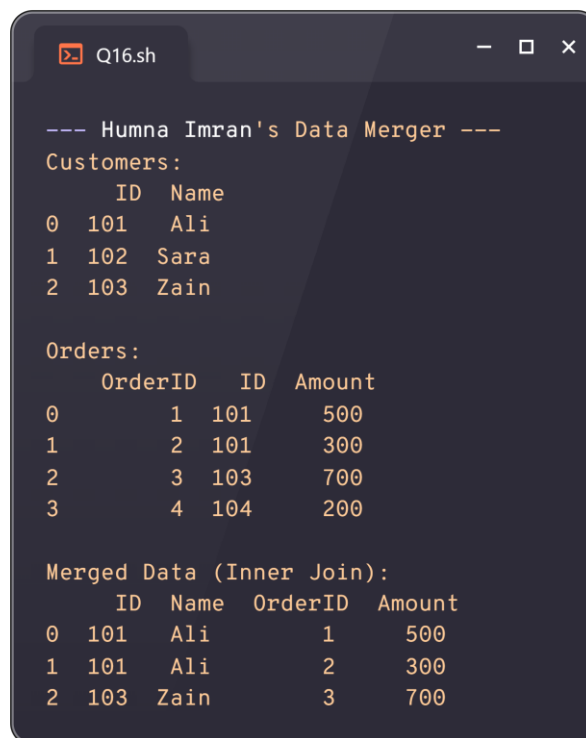
orders = pd.DataFrame({
    'OrderID': [1, 2, 3, 4],
    'ID': [101, 101, 103, 104], # 104 is a non-existent customer
    'Amount': [500, 300, 700, 200]
})

print("Customers:\n", customers)
print("\nOrders:\n", orders)

# 2. Inner Join (Only matching IDs)
merged_df_humna = pd.merge(customers, orders, on='ID', how='inner')
print("\nMerged Data (Inner Join):\n", merged_df_humna)

```

output



```

Q16.sh

--- Humna Imran's Data Merger ---
Customers:
   ID  Name
0  101  Ali
1  102  Sara
2  103  Zain

Orders:
   OrderID  ID  Amount
0         1  101    500
1         2  101    300
2         3  103    700
3         4  104    200

Merged Data (Inner Join):
   ID  Name  OrderID  Amount
0  101  Ali         1    500
1  101  Ali         2    300
2  103  Zain         3    700

```

Question # 39:

String Manipulation. A DataFrame contains names in a messy format (mixed case, extra spaces). Write a program to clean the names by converting them to Title Case and stripping whitespace.

Code:

```
import pandas as pd
print("--- Humna Imran's Text Formatter ---")

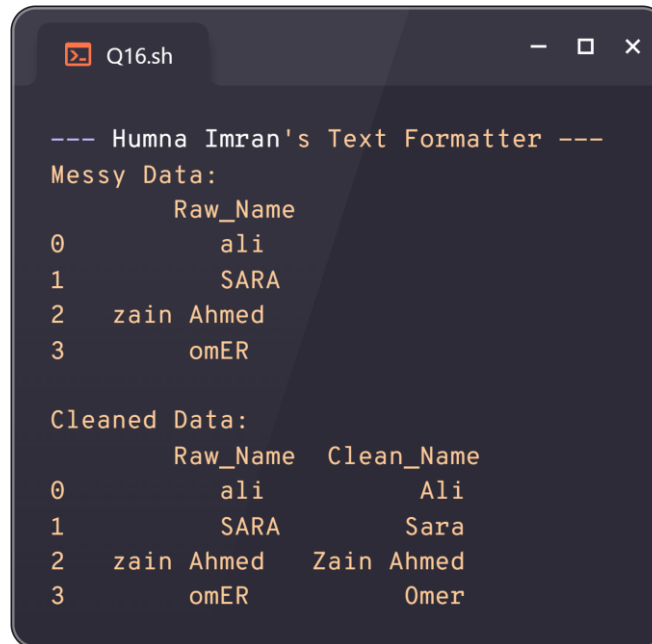
data = {'Raw_Name': [' ali ', 'SARA', 'zain Ahmed ', ' omER ']}
df_text_humna = pd.DataFrame(data)

print("Messy Data:\n", df_text_humna)

# 1. String Methods
df_text_humna['Clean_Name'] =
df_text_humna['Raw_Name'].str.strip().str.title()

print("\nCleaned Data:\n", df_text_humna)
```

output



```
Q16.sh

--- Humna Imran's Text Formatter ---
Messy Data:
   Raw_Name
0      ali
1      SARA
2  zain Ahmed
3      omER

Cleaned Data:
   Raw_Name  Clean_Name
0      ali      Ali
1      SARA      Sara
2  zain Ahmed  Zain Ahmed
3      omER      Omer
```

Question # 40:

DateTime Operations. Create a DataFrame with a column of date strings. Convert this column to DateTime objects. Extract and create new columns for Year, Month, and Day.

Code:

```
import pandas as pd
print("--- Humna Imran's Date Parser ---")

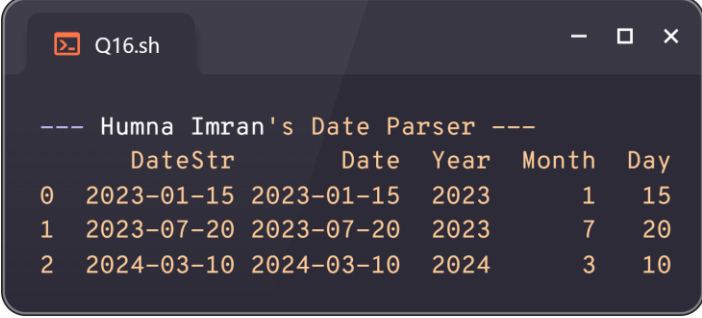
# 1. Create DataFrame
df_dates_humna = pd.DataFrame({'DateStr': ['2023-01-15', '2023-07-20',
'2024-03-10']})

# 2. Convert to DateTime
df_dates_humna['Date'] = pd.to_datetime(df_dates_humna['DateStr'])

# 3. Extract Components
df_dates_humna['Year'] = df_dates_humna['Date'].dt.year
df_dates_humna['Month'] = df_dates_humna['Date'].dt.month
df_dates_humna['Day'] = df_dates_humna['Date'].dt.day

print(df_dates_humna)
```

output



```
--- Humna Imran's Date Parser ---
  DateStr      Date  Year  Month  Day
0 2023-01-15 2023-01-15 2023     1   15
1 2023-07-20 2023-07-20 2023     7   20
2 2024-03-10 2024-03-10 2024     3   10
```


LAB # 9

Matplotlib - Basic Plotting

Question # 41:

Simple Line Plot. Plot the relationship between study hours and exam scores.

Data: Hours [1, 2, 3, 4, 5], Scores [50, 60, 65, 75, 85].

Add titles and labels.

Code:

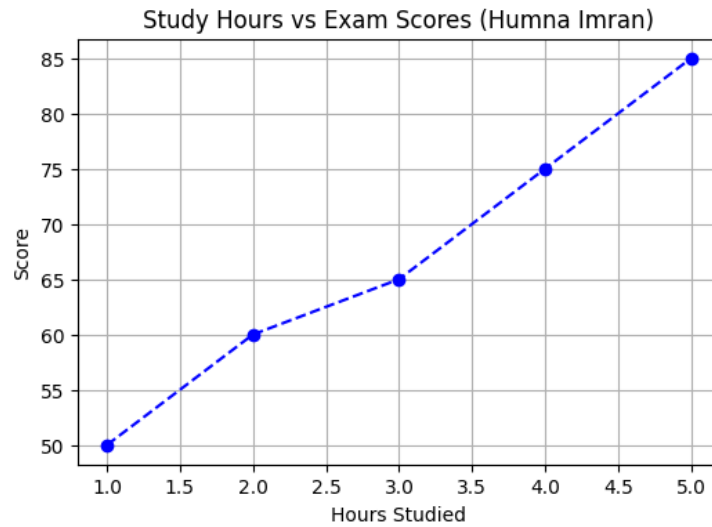
```
import matplotlib.pyplot as plt
print("--- Humna Imran's Line Plot ---")

hours_humna = [1, 2, 3, 4, 5]
scores_humna = [50, 60, 65, 75, 85]

plt.figure(figsize=(6, 4))
plt.plot(hours_humna, scores_humna, marker='o', color='blue',
linestyle='--')

plt.title("Study Hours vs Exam Scores (Humna Imran)")
plt.xlabel("Hours Studied")
plt.ylabel("Score")
plt.grid(True)
plt.show()
```

output



Question # 42:

Bar Chart. Compare the sales of three different products: ['Laptop', 'Mouse', 'Keyboard'] with sales figures [50, 120, 80].

Code:

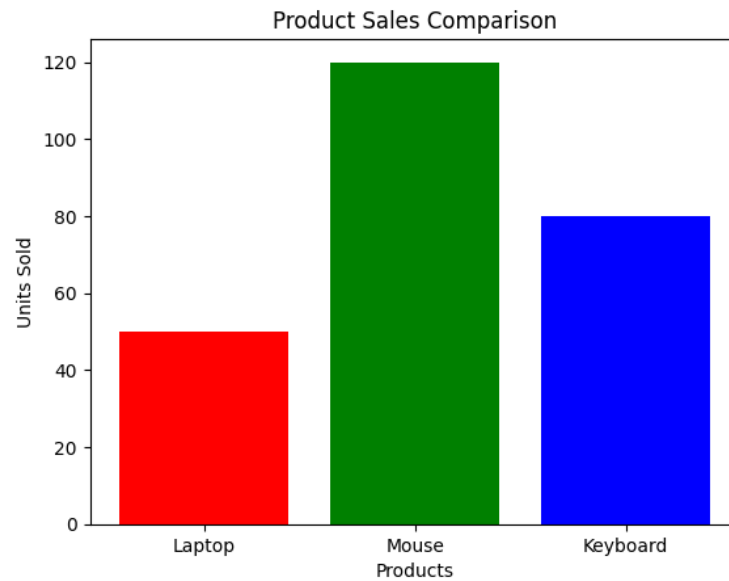
```
import matplotlib.pyplot as plt
print("--- Humna Imran's Bar Chart ---")

products = ['Laptop', 'Mouse', 'Keyboard']
sales_humna = [50, 120, 80]

plt.bar(products, sales_humna, color=['red', 'green', 'blue'])

plt.title("Product Sales Comparison")
plt.xlabel("Products")
plt.ylabel("Units Sold")
plt.show()
```

output



Question # 43:

Scatter Plot. Visualizing correlation using random data points. Generate 20 random points for X and Y coordinates and plot them as a scatter chart.

Code:

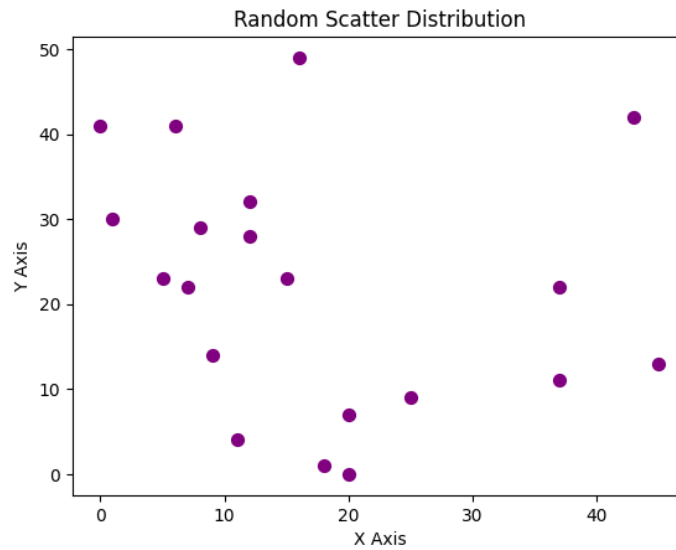
```
import matplotlib.pyplot as plt
import numpy as np
print("--- Humna Imran's Scatter Plot ---")

np.random.seed(1)
x_vals_humna = np.random.randint(0, 50, 20)
y_vals_humna = np.random.randint(0, 50, 20)

plt.scatter(x_vals_humna, y_vals_humna, color='purple', s=50) # s is
marker size

plt.title("Random Scatter Distribution")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

output



Question # 44:

Pie Chart. Visualize the budget allocation of a project.

Categories: ['Development', 'Marketing', 'Server', 'Design']

Budget: [40, 30, 20, 10]

Code:

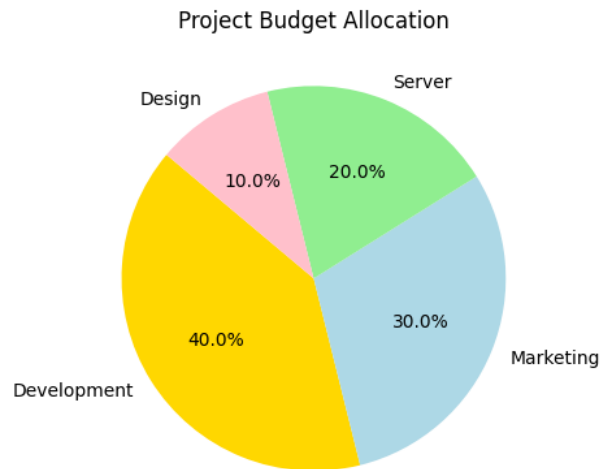
```
import matplotlib.pyplot as plt
print("---- Humna Imran's Pie Chart ----")

categories = ['Development', 'Marketing', 'Server', 'Design']
budget_humna = [40, 30, 20, 10]
colors = ['gold', 'lightblue', 'lightgreen', 'pink']

plt.pie(budget_humna, labels=categories, colors=colors,
autopct='%1.1f%%', startangle=140)

plt.title("Project Budget Allocation")
plt.show()
```

output



Question # 45:

Histogram. Generate an array of 100 random ages between 18 and 60. Plot a histogram to show the frequency distribution of age groups.

Code:

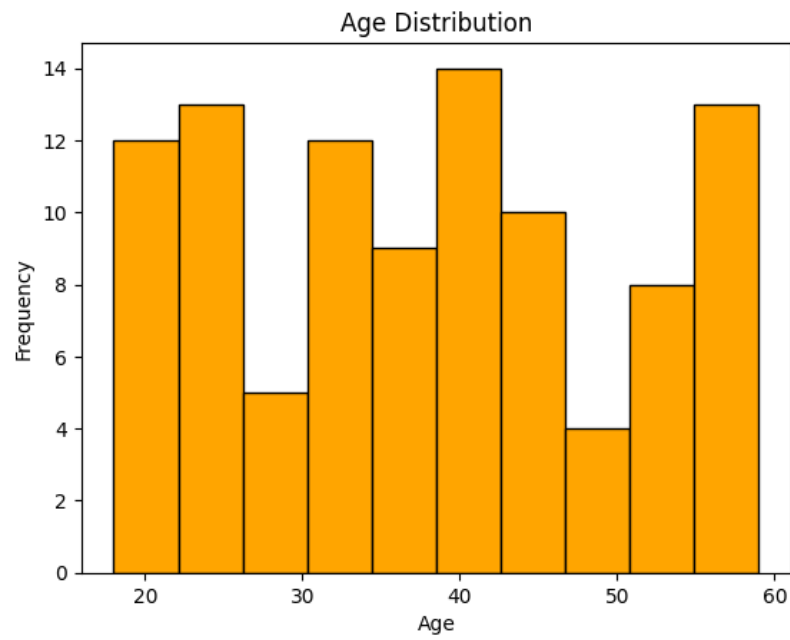
```
import matplotlib.pyplot as plt
import numpy as np
print("--- Humna Imran's Histogram ---")

np.random.seed(42)
ages_humna = np.random.randint(18, 60, 100)

plt.hist(ages_humna, bins=10, color='orange', edgecolor='black')

plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

output



LAB # 10

Matplotlib - Advanced Visualizations

Question # 46:

Subplots. Create a figure with 2 rows and 1 column of plots.

- Top plot: A sine wave.
 - Bottom plot: A cosine wave.
- Range for x is 0 to 10.

Code:

```
import matplotlib.pyplot as plt
import numpy as np
print("--- Humna Imran's Subplots ---")

x = np.linspace(0, 10, 100)
y1_humna = np.sin(x)
y2_humna = np.cos(x)

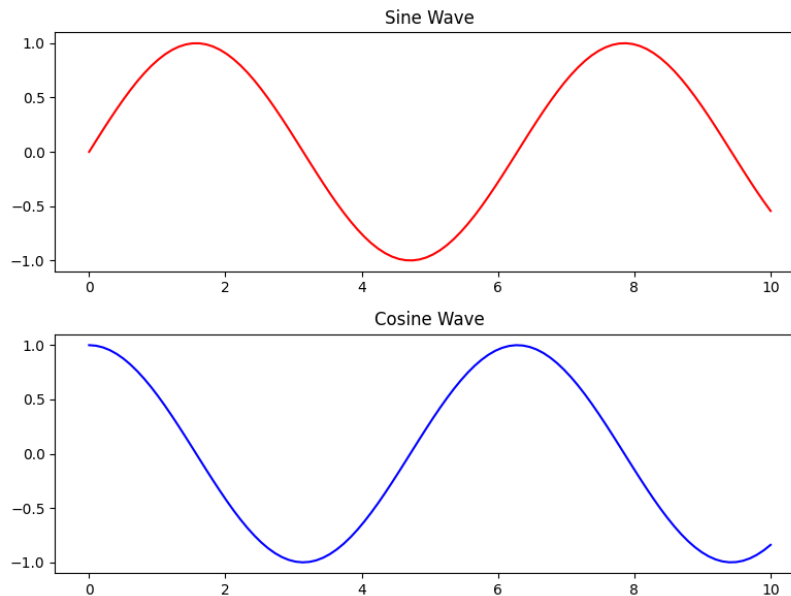
# Create 2 subplots
plt.figure(figsize=(8, 6))

plt.subplot(2, 1, 1) # (rows, cols, index)
plt.plot(x, y1_humna, 'r')
plt.title("Sine Wave")

plt.subplot(2, 1, 2)
plt.plot(x, y2_humna, 'b')
plt.title("Cosine Wave")

plt.tight_layout()
plt.show()
```

output



Question # 47:

Customizing Legends and Styles. Plot two lines on the same graph:

- Line 1: $y = x^2$ (Dashed Line, Label "Squared")
 - Line 2: $y = x^3$ (Solid Line, Label "Cubed")
- Add a legend and grid.

Code:

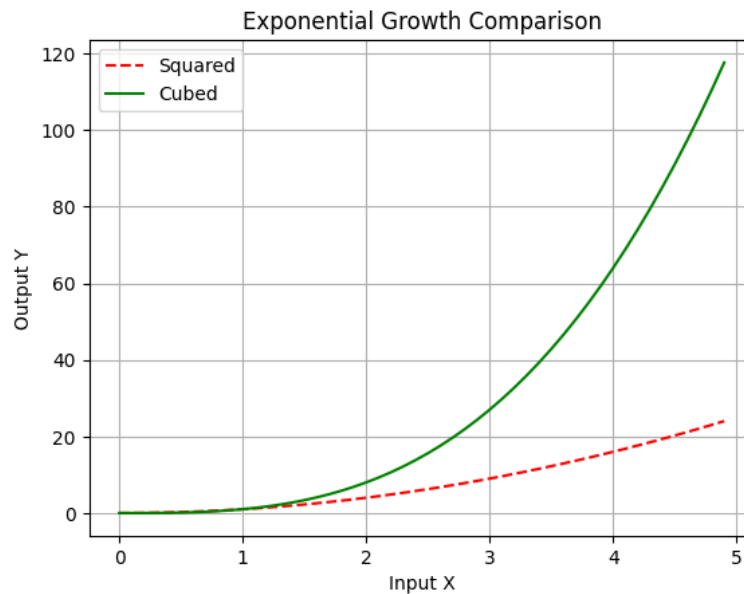
```
import matplotlib.pyplot as plt
import numpy as np
print("--- Humna Imran's Custom Plot ---")

x = np.arange(0, 5, 0.1)

plt.plot(x, x**2, 'r--', label='Squared') # Red Dashed
plt.plot(x, x**3, 'g-', label='Cubed')    # Green Solid

plt.title("Exponential Growth Comparison")
plt.xlabel("Input X")
plt.ylabel("Output Y")
plt.legend()
plt.grid(True)
plt.show()
```


output



Question # 48:

Box Plot. Visualize the spread of data and detect outliers. Create three lists of random numbers representing test scores from three different classes and plot them side-by-side using a boxplot.

Code:

```
import matplotlib.pyplot as plt
import numpy as np
print("--- Humna Imran's Box Plot ---")

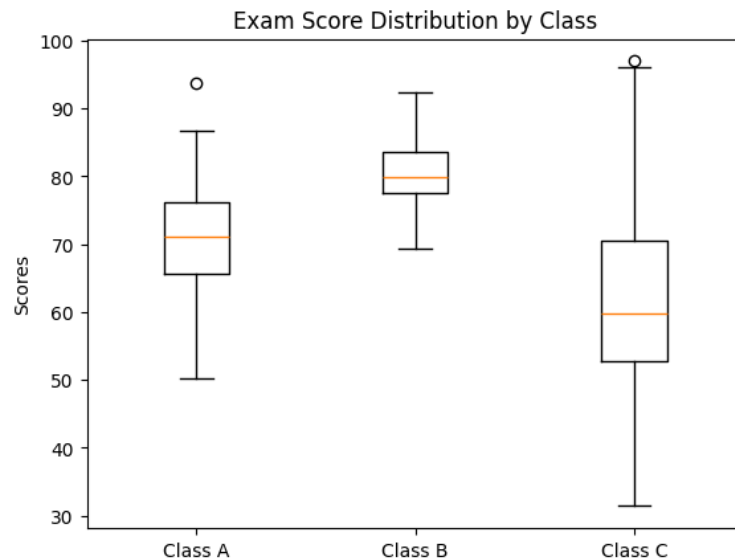
np.random.seed(10)
class_A = np.random.normal(70, 10, 50)
class_B = np.random.normal(80, 5, 50)
class_C = np.random.normal(60, 15, 50) # High variance

data_humna = [class_A, class_B, class_C]

plt.boxplot(data_humna, labels=['Class A', 'Class B', 'Class C'])

plt.title("Exam Score Distribution by Class")
plt.ylabel("Scores")
plt.show()
```

output



Question # 49:

Stacked Bar Chart. Compare the gold and silver medals won by three countries.

- Country A: 10 Gold, 5 Silver
- Country B: 15 Gold, 10 Silver
- Country C: 8 Gold, 8 Silver

Code:

```
import matplotlib.pyplot as plt
print("--- Humna Imran's Stacked Bar ---")

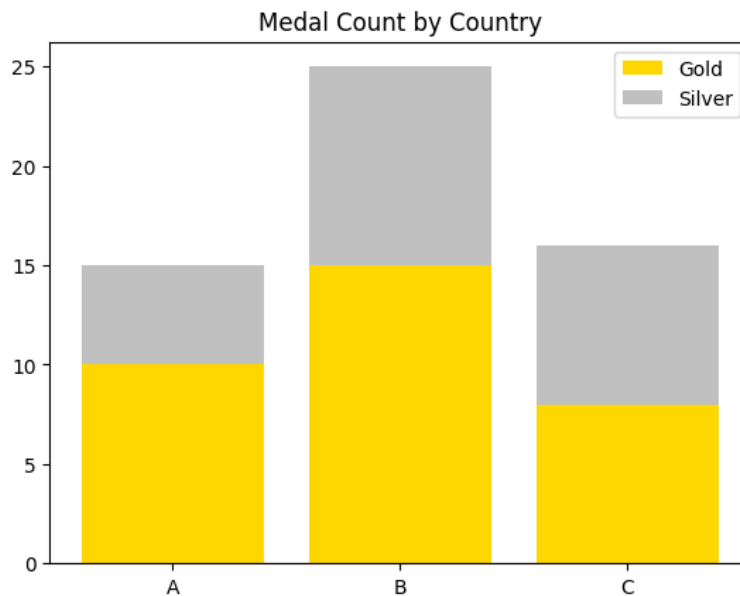
countries = ['A', 'B', 'C']
gold_humna = [10, 15, 8]
silver_humna = [5, 10, 8]

# Plot Gold first
plt.bar(countries, gold_humna, label='Gold', color='gold')

# Plot Silver on top of Gold (bottom=gold)
plt.bar(countries, silver_humna, bottom=gold_humna, label='Silver',
color='silver')

plt.title("Medal Count by Country")
plt.legend()
plt.show()
```

output



Question # 50:

Saving Plots. Write a script to generate a simple plot of $y=x$ and save it as an image file named humna_plot.png instead of just displaying it.

Code:

```
import matplotlib.pyplot as plt
print("--- Humna Imran's File Saver ---")

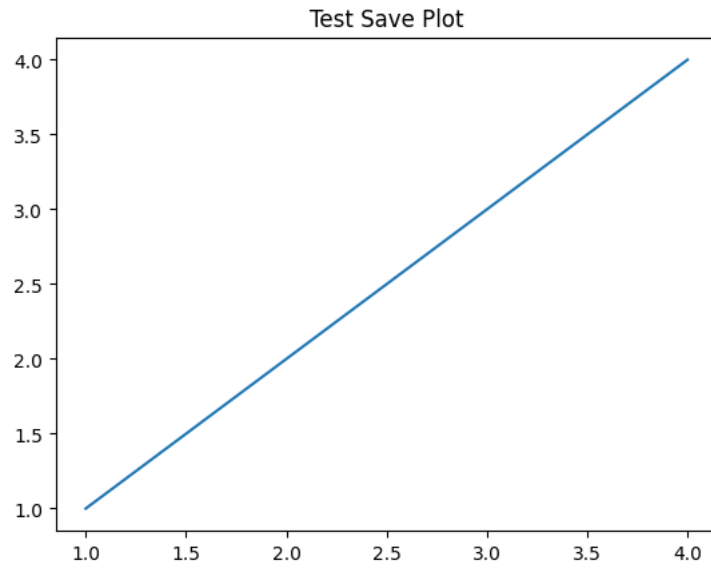
x = [1, 2, 3, 4]
y = [1, 2, 3, 4]

plt.plot(x, y)
plt.title("Test Save Plot")

# Save file
plt.savefig("humna_plot.png")
print("Plot saved as 'humna_plot.png' successfully.")
```

output

```
--- Humna Imran's File Saver ---
Plot saved as 'humna_plot.png' successfully.
```



LAB # 11

Seaborn - Statistical Data Visualization

Question # 51:

Patient Stay Analysis (Healthcare). You are given a dataset containing age, disease_type, and hospital_stay_days. Write a Python program using Seaborn to plot the distribution of hospital stay days and identify which disease type corresponds to longer stays.

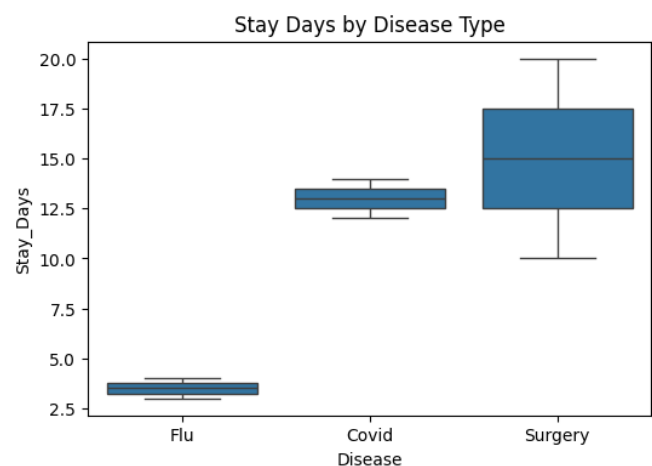
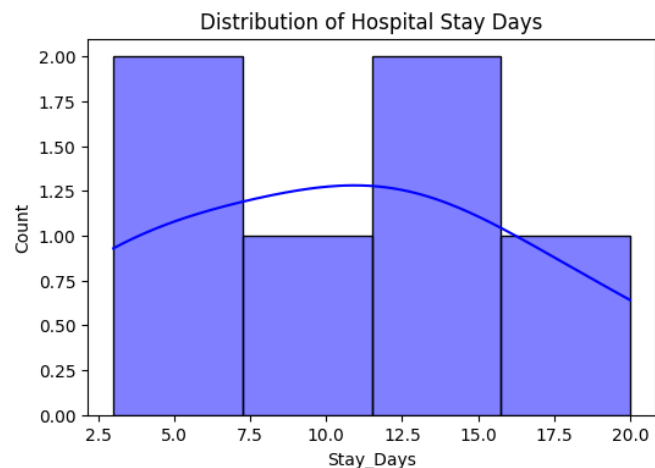
Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
print("--- Humna Imran's\nHealthcare Analysis ---")

# Mock Dataset
data_humna = {
    'Disease': ['Flu', 'Covid',
               'Flu', 'Surgery', 'Surgery',
               'Covid'],
    'Stay_Days': [3, 14, 4, 10, 20, 12]}
df = pd.DataFrame(data_humna)

# 1. Distribution Plot
plt.figure(figsize=(6, 4))
sns.histplot(df['Stay_Days'],
             kde=True, color='blue')
plt.title("Distribution of\nHospital Stay Days")
plt.show()

# 2. Boxplot Comparison
plt.figure(figsize=(6, 4))
sns.boxplot(x='Disease',
            y='Stay_Days', data=df)
plt.title("Stay Days by Disease Type")
plt.show()
```



Question # 52:

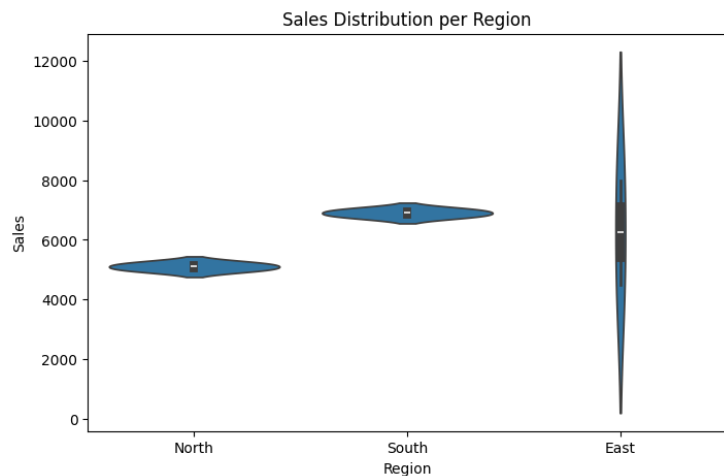
Sales Distribution Comparison (Business). A retail dataset contains Region and Monthly_Sales. Write code to visualize sales distribution per region using Seaborn to detect variability.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
print("--- Humna Imran's  
Business Stats ---")
```

```
# Mock Dataset
data_humna = pd.DataFrame({
    'Region': ['North',
               'North', 'South', 'South',
               'East', 'East'],
    'Sales': [5000, 5200,
              7000, 6800, 4500, 8000] #
    East has high variability
})
```

```
plt.figure(figsize=(8, 5))
sns.violinplot(x='Region', y='Sales', data=data_humna)
plt.title("Sales Distribution per Region")
plt.show()
```



Question # 53:

Student Performance Correlation (Education). A dataset contains Study_Hours, Attendance, and Final_Score. Using Seaborn, create a pair plot and a heatmap to interpret which feature affects the score most.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
print("--- Humna Imran's Education Analysis ---")
```

```
# Mock Dataset
df_edu_humna = pd.DataFrame({
    'Study_Hours': [2, 4, 6, 8, 10],
```

```

    'Attendance': [80, 85, 90, 95, 100],
    'Score': [60, 70, 80, 85, 95]
})

```

1. Pair Plot

```

sns.pairplot(df_edu_humna)
plt.suptitle("Feature Pair Plot", y=1.02)
plt.show()

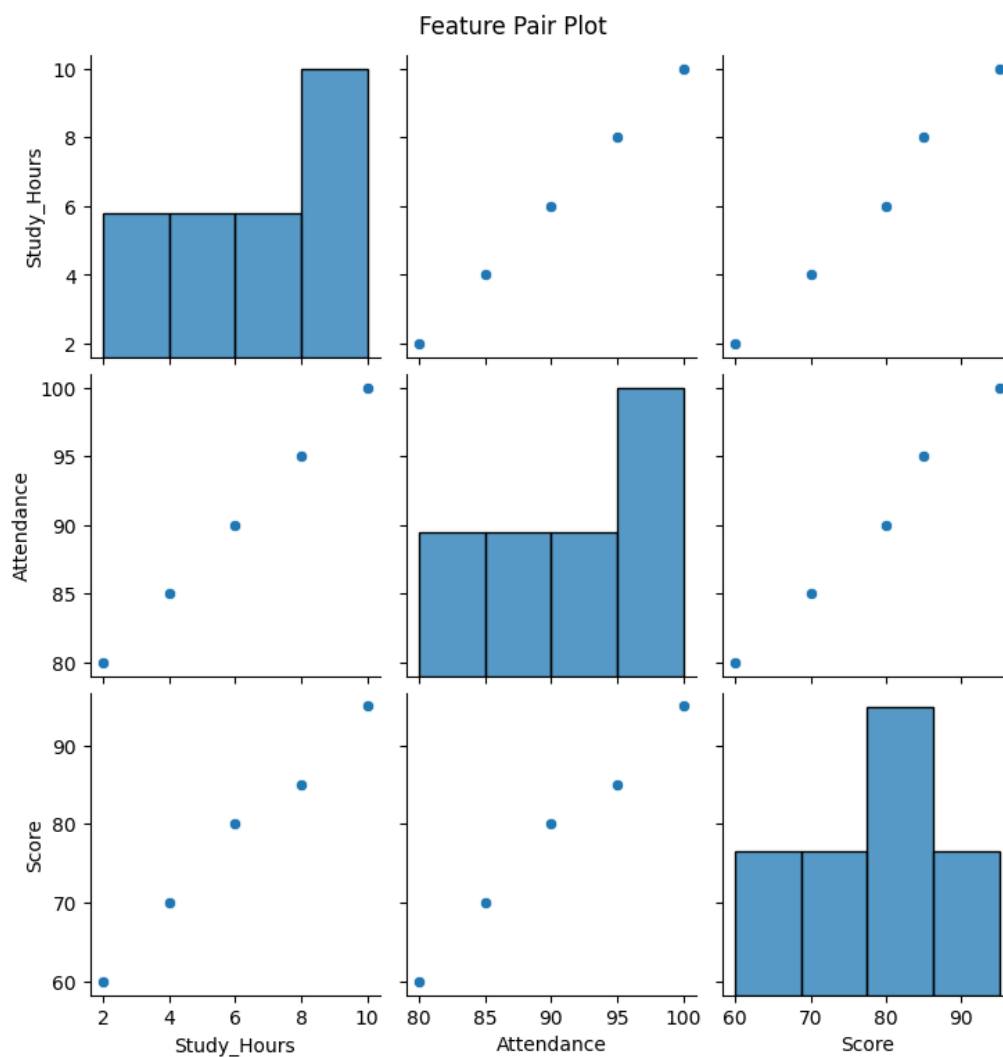
```

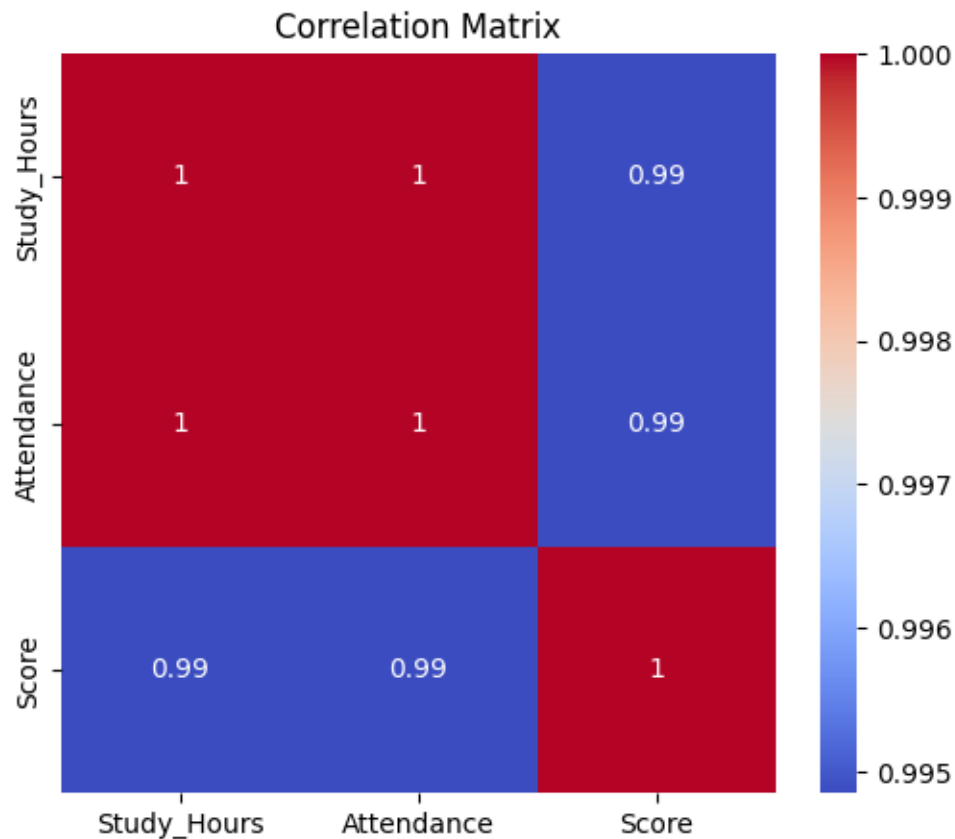
2. Correlation Heatmap

```

plt.figure(figsize=(6, 5))
sns.heatmap(df_edu_humna.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()

```





Question # 54:

Customer Segmentation Visualization. Customer data includes Age, Annual_Income, and Spending_Score. Visualize relationships using a scatter plot, color-coding points based on the spending score.

Code:

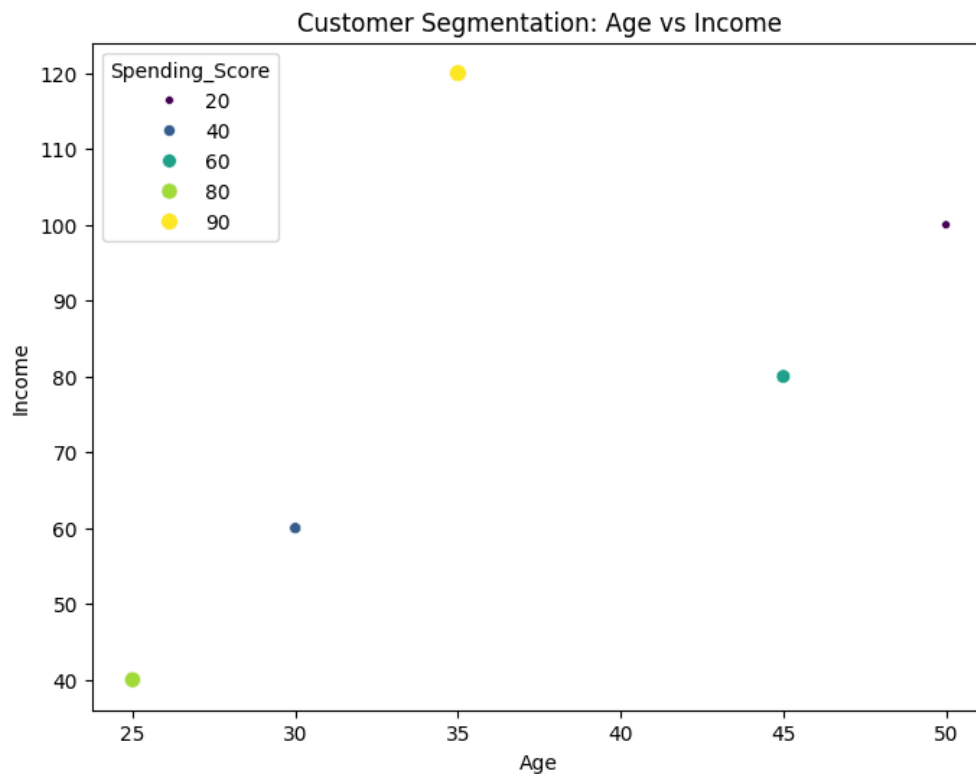
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
print("--- Humna Imran's Customer Segments ---")

# Mock Dataset
df_cust_humna = pd.DataFrame({
    'Age': [25, 30, 45, 50, 35],
    'Income': [40, 60, 80, 100, 120],
    'Spending_Score': [80, 40, 60, 20, 90],
    'Segment': ['High', 'Medium', 'Medium', 'Low', 'High']
})

plt.figure(figsize=(8, 6))
```



```
sns.scatterplot(x='Age', y='Income', hue='Spending_Score',
size='Spending_Score', data=df_cust_humna, palette='viridis')
plt.title("Customer Segmentation: Age vs Income")
plt.show()
```



Question # 55:

Model Result Visualization. You trained a classifier and obtained `y_true` and `y_pred`. Write code using Seaborn to plot a confusion matrix heatmap to visualize the accuracy.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
print("---- Humna Imran's Model Evaluation ----")

y_true_humna = [0, 1, 0, 1, 0, 1, 1, 0]
y_pred_humna = [0, 0, 0, 1, 0, 1, 1, 1]

# Generate Confusion Matrix
cm = confusion_matrix(y_true_humna, y_pred_humna)
```

```
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Class 0', 'Class 1'], yticklabels=['Class 0', 'Class 1'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title("Model Confusion Matrix")
plt.show()
```

